

クラス関係図の設計原則

本ドキュメントは、`class_overview_diagram.md` の設計意図と、その背景にある専門的な概念を説明する。

1. 修正の経緯

修正内容

項目	修正前	修正後
図の位置づけ	「クラス設計図」	「クラス関係図」
スコープ	明記なし	公開APIのみ、プライベートは省略と明記
SSOT	なし	本図を正とすると明記

修正理由

- 将来的にプライベートメソッドを含む詳細図が作成される可能性がある
- 「関係図」であることを明確にし、スコープを限定する必要がある
- 各クラスをサブシステムとした「システムの関係図」という位置づけを明確化

2. SSOT (Single Source of Truth)

定義

情報の正規化原則。同じ情報が複数箇所に存在すると、更新漏れや矛盾が発生する。

背景

チーム開発では、設計情報が複数のドキュメントに散在しがち。例えば：

- クラス図
- インターフェース仕様書
- オンボーディング資料
- コードコメント

これらが矛盾すると、メンバーは「どれが正しいのか」を判断できず、誤った実装が生まれる。

適用

`class_overview_diagram.md` に「本図を正とする」と明記することで：

- 矛盾発見時の判断基準が明確になる
- 更新すべき箇所が一元化される
- 議論の出発点が統一される

3. UML クラス図の抽象度レベル

Martin Fowler の3レベル ^[1]

レベル	目的	記載内容	対象読者
Conceptual (概念)	ドメイン理解	クラス名、主要な関係のみ	全員
Specification (仕様)	インターフェース定義	公開API、型、関係性	設計者、実装者
Implementation (実装)	コード設計	プライベートメンバ含む全詳細	実装者

適用

`class_overview_diagram.md` は **Specification レベル** として設計。

- 公開API (+) と関係性を記載
- プライベートメソッド (-) は省略
- 実装詳細は `.hpp` / `.cpp` で定義

4. 関心の分離 (Separation of Concerns)

定義

各成果物は単一の関心に集中すべきという設計原則。

背景

一つのドキュメントに複数の関心を詰め込むと：

- 本来の目的が埋もれる
- 更新が困難になる
- 読み手が必要な情報を見つけにくい

適用

成果物	関心
<code>class_overview_diagram.md</code>	クラス間の 関係性 と公開 API
<code>.hpp</code> ファイル	実装の インターフェース 詳細
<code>.cpp</code> ファイル	実装 の詳細

図に全てを詰め込まず、各成果物の責務を明確に分離する。

5. インターフェースと実装の分離

GoF の設計原則 ^[2]

"Program to an interface, not an implementation."
(実装ではなくインターフェースに対してプログラムせよ)

背景

Gang of Four (GoF) は、1994年の著書「Design Patterns」で23のデザインパターンを体系化した。その中で、再利用可能な設計の原則として「インターフェースへのプログラミング」を提唱。

この原則の意図：

- クライアント ^[3] は具体的な実装ではなく、抽象（インターフェース）に依存すべき
- 実装の詳細を隠蔽することで、変更の影響範囲を限定できる
- テスト容易性が向上する（モック/スタブの差し替えが可能）

適用

プライベートメソッドは「実装の詳細」であり、他クラスから見えない。

関係図にプライベートを記載すると：

- 本来隠蔽すべき情報が露出する
- 読み手が不適切な依存を持つ可能性がある
- 実装変更時に図の更新が必要になる

したがって、関係図には公開APIのみを記載し、プライベートは省略する。

6. ドキュメントのスコープ管理

問題

スコープを明示しないと、読み手は「何が載っていて何が載っていないか」を判断できない。

例：

- 「このメソッドが図にない。設計漏れ？」
- 「プライベートメソッドを追加すべき？」

対策

冒頭に「スコープ」を明記し、記載範囲を限定する。

> ****スコープ****: クラス間の関係性と公開API（`^+^`）のみ記載。
> プライベートメソッド（`^-^`）は省略。詳細は実装フェーズで決定。

これにより：

- 「載っていない」ことが「忘れた」のか「意図的」なかが明確
- 将来の拡張時にスコープ変更を議論できる

- 新規メンバーが図の読み方を理解できる

7. アーキテクチャ文書 vs 詳細設計文書

分類

種類	対象読者	抽象度	更新頻度	例
アーキテクチャ	全チーム、ステークホルダー	高	低	<code>class_overview_diagram.md</code>
詳細設計	実装担当者	低	高	<code>.hpp</code> 、実装仕様書

適用

`class_overview_diagram.md` は **アーキテクチャ文書** として位置づける。

- 全員が共通認識を持つための「地図」
- 全ての道路名（プライベートメソッド）を載せる必要はない
- 大きな構造変更時のみ更新

8. まとめ

概念	本図への適用
SSOT	「本図を正とする」と明記
UML Specification レベル	公開APIのみ、実装詳細は省略
関心の分離	関係性と公開APIに集中
インターフェース分離（GoF）	プライベートは隠蔽
スコープ管理	記載範囲を冒頭に明記
アーキテクチャ文書	全員の共通認識用「地図」

参考文献

書籍/資料	著者	該当トピック
UML Distilled (3rd Ed.)	Martin Fowler	UML の抽象度レベル
Design Patterns	GoF (Gamma et al.)	インターフェースへのプログラミング
Clean Architecture	Robert C. Martin	関心の分離、依存性の方向

関連ドキュメント

ドキュメント	内容
class_overview_diagram.md	本原則を適用したクラス関係図
design.md	全体設計・責務分割
interface.md	層間API契約

- Martin Fowler, "UML Distilled: A Brief Guide to the Standard Object Modeling Language", 3rd Edition, Addison-Wesley, 2003. [↩](#)
- Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley, 1994. 通称「GoF本」。 [↩](#)
- ここでの「クライアント」は IRC クライアント（irssi 等）ではなく、オブジェクト指向設計における「利用する側のコード」を指す。例: `CommandDispatcher` は `Client` クラスの「クライアント」。[↩](#)